

PocketRole ユーザーズガイド

更新日: 2026-05-02

このガイドは `docs/users\_manual/` 配下を正本とする、PocketRole の運用者向けユーザーズガイドです。

主な対象は「ローカルで設定し、story を投入し、engine を動かし、必要なら web viewer まで確認する人」です。

新規ユーザーは、まず repo 直下の `START\_HERE.md` を読んでください。

このガイドは「初回成功のあとに、継続運用や設定変更を行う人」向けです。

AI エージェントに導入や設定作業を代行させる場合は、あわせて

`docs/upgrade/version2/AI\_AGENT\_OPERATIONS\_MEMO.md` を読ませてください。

このガイドは人間向け、`AI\_AGENT\_OPERATIONS\_MEMO.md` は AI エージェント向けの安全運用メモです。

1. PocketRole とは

PocketRole は、キャラクターが会話・移動・感情変化を続けるストーリー生成アプリです。

- ローカル側の正本は SQLite DB です
- 公開 viewer 側の正本は hosted 環境の `.dat` ログです
- そのため、ローカル DB にログがあっても、Web 送信が成功するまでは公開 viewer には出ません

この違いを理解しておくと、運用時の切り分けがかなり楽になります。

2. できること

PocketRole では主に次のことができます。

1. story 定義 YAML を検証する
2. 新しい story を DB に投入する
3. 進行中 story の定義だけを安全に更新する
4. engine を起動して会話を継続生成する
5. admin の local viewer で表情画像付きの会話を読む
6. sample story から自作 story を作り始める
7. `/tmp` の作業 DB で短時間の smoke 実行をする
8. ログや小説出力を Markdown / text で書き出す
9. 必要に応じて hosted viewer に公開する

3. 利用前に知っておくべきこと

story の新規投入と更新は別コマンド

- 新規 story は `import\_story`
- 進行中 story の定義更新は `update\_story`
- 進行中 story に `import\_story --force-replace` を使うと再構築扱いなので、通常運用では避けてください

smoke 実行は本番 DB でも viewer 確認でもない

- `smoke\_engine\_start` は `/tmp` にコピーした DB で少数ターンだけ回します

- 本番 DB を汚さずに確認できる代わりに、public viewer 更新確認には使えません

公開 viewer の正本は local DB ではない

公開 viewer は `receiver.php` に送られた `.dat` ログを読んでいます。
そのため、viewer の確認は DB の中身だけではなく、`status.php` や `api.php?action=latest` でも行います。

4. 事前準備

必要なもの

- Python 3.11 以上
- 仮想環境
- SQLite を使えるローカル環境
- OpenAI API を使うなら API key
- Ollama を使うならローカル Ollama 環境
- Manual PDF を自分でやり直す場合は `.venv` の Python 環境と current の PDF 再生成手順

ディレクトリに移動

```
```bash
cd /home/test/Desktop/project-test/pocketrole_project_upgrade1_bbf/pocketrole
```
```

仮想環境の作成と依存インストール

```
```bash
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```
```

5. 初回セットアップ

5-0. まずどのルートで始めるか

新規ユーザーには、まず `START\_HERE.md` の OpenAI API ルートを推奨します。
理由は、Ollama 導入よりも初回成功までが短いからです。

推奨順:

1. OpenAI API で `ankoku\_gakuen` を一度動かす
2. 必要なら Ollama に切り替える
3. 自作 story / 自作キャラへ差し替える

このガイドでは、その後の継続運用に必要な詳細を扱います。

5-1. config.yaml を作る

まず基本設定をコピーします。

```
```bash
cp config.yaml.example config.yaml
```
```

`config.yaml.example` は base runtime の雛形ですが、scene-aware な story-emergence 機能は `**opt-in**` です。

特に次の feature flag 群は、既定では `enabled: false` が多く、コピーしただけでは新経路は有効になりません。

- `scene\_management`
- `participation\_planner`
- `story\_hooks`
- `character\_drives`
- `relationship\_dynamics`
- `quality\_guard`
- `growth\_engine`

まずは `config.yaml` を作り、そのうえで必要な runtime flag だけを明示的に有効化してください。

5-2. .env を必要に応じて作る

この repo には `.env.example` はありません。必要な変数だけ自分で作ります。

最小例:

```
```.env
OLLAMA_BASE_URL=http://localhost:11434
OPENAI_API_KEY=
ANTHROPIC_API_KEY=
GEMINI_API_KEY=
DEEPSEEK_API_KEY=
```
```

`.env` は shell 互換ではなく、単純な `KEY=VALUE` 形式で読まれます。
引用符や行末コメントは付けないでください。

5-2a. local 設定ファイルは example と分ける

次のファイルは environment-local です。

- `.env`
- `config/llm\_runtime.local.yaml`
- `config/web\_post\_targets.local.yaml`
- `db/pocketrole.db`

配布用テンプレートや sample story と混ぜず、上書き時は example 側と取り違えないでください。

5-3. Web 投稿を使うなら投稿先設定ファイルを作る

Web viewer へ反映したい場合、投稿先は `.env` ではなく
`config/web\_post\_targets.local.yaml` で story ごとに設定します。

まず空の設定ファイルを作ります。

```
```.bash
python -m tools.init_web_post_targets --stories ankoku_gakuen
```
```

その後、`config/web\_post\_targets.local.yaml` を編集します。

例:

```

```yaml
ankoku_gakuen:
 enabled: true
 receiver_url: "https://example.com/chururun/receiver.php"
 auth_token: "your-secret-token"
```

```

このファイルは gitignore 対象です。

Web サーバー側の `web/config/auth\_token.txt` には、同じ token を 1 行で置いてください。

5-4. Ollama を使う場合

これは初回成功の最短ルートではなく、**動作確認後にコストを抑えて試行回数を増やしたい場合の第 2 ルート** です。

接続情報は `config.yaml` と `.env` に置きますが、**日常運用でどの LLM を使うかの切替点は `config/llm\_runtime.local.yaml`** です。

まずモデルを用意します。

```

```bash
ollama pull minstral-3:14b
```

```

ローカル Ollama が `11434` 以外で動いている場合は、`OLLAMA\_BASE\_URL` または `config.yaml` の `llm.providers.ollama.base\_url` を合わせてください。

5-5. LLM の切替を 1 ファイルで管理する

Ollama / OpenAI / Claude などの**日常運用切替**は、story YAML や `config.yaml` ではなく `config/llm\_runtime.local.yaml` で行います。

まず example を元に作ります。

```

```bash
cp config/llm_runtime.example.yaml config/llm_runtime.local.yaml
```

```

最小例:

```

```yaml
active_profile: "openai_nano"

profiles:
 ollama_local:
 provider: "ollama"
 model: "minstral-3:14b"

 openai_nano:
 provider: "openai"
 model: "gpt-5.4-nano"

story_overrides:
 ankoku_gakuen_mystery: "ollama_local"
```

```

意味は次のとおりです。

- `active\_profile`

- 全 story の既定 runtime provider/model
- `profiles`
 - 切替候補の一覧
- `story\_overrides`
 - 特定 story だけ別 profile を使いたいときの上書き

優先順位は次です。

1. `story\_overrides`
2. `active\_profile`

つまり、通常運用では `world\_config.yaml` や `config.yaml` をいじらず、この 1 ファイルだけで Ollama と OpenAI を切り替えられます。

API key などの secret は引き続き `.env` に置きます。

5-5a. chapter / director を使う story では追加 import が必要

`chapters.yaml` と `director.yaml` を持つ story では、`world\_config.yaml` / `characters.yaml` の import/update だけでは反映が完了しません。

必要に応じて次も実行します。

```
```bash
python -m tools.import_chapters --story ankoku_gakuen --db db/pocketrole.db
python -m tools.import_director --story ankoku_gakuen --db db/pocketrole.db
```
```

切替時は次を使います。

```
```bash
python -m tools.activate_chapter --story ankoku_gakuen --chapter <chapter_id> --turn <turn_number> --db db/pocketrole.db
python -m tools.switch_director --story ankoku_gakuen --persona <persona_id> --turn <turn_number> --db db/pocketrole.db
```
```

5-6. story-emergence runtime を使う場合

scene-aware な物語制御を実運用で使いたい場合は、`config.yaml` に次のようなセクションを足します。

```
```yaml
scene_management:
 enabled: true
 max_active_scenes_per_story: 3

participation_planner:
 enabled: true
 max_focus_characters: 2
 max_support_characters: 2

story_hooks:
 enabled: true
 max_open_hooks: 10

relationship_dynamics:
 enabled: true
 trust_delta_scale: 0.1

quality_guard:
```

```

 enabled: true
 max_group_chars: 180
 max_reply_chars: 140
 max_monologue_chars: 110
 max_abstract_token_occurrences: 3
 max_repeated_ngram_occurrences: 2

growth_engine:
 enabled: true
 rolling_window_turns: 20
 immediate_commit_identity_threshold: 0.85
...

```

これらを入れない場合、code 上に新機能があっても実運転は旧来の全員発話寄りの経路のままになりえます。

## ## 6. story の構成

story は通常、`stories/<story\_id>/` に置きます。

最低限必要なのは次の 2 ファイルです。

- `world\_config.yaml`
- `characters.yaml`

例:

```

```text
stories/
  ankoku_gakuen/
    world_config.yaml
    characters.yaml
...

```

7. 新規 story を開始する手順

7-1. YAML を検証する

```

```bash
python -m tools.validate_story stories/ankoku_gakuen
...

```

ここでは主に次を確認します。

- `story\_id` や `place\_id` の形式
- `day\_type`, `zone` の妥当性
- 時刻や日付の形式
- place 参照整合

### ### 7-2. DB に投入する

```

```bash
python -m tools.import_story stories/ankoku_gakuen --db db/pocketrole.db
...

```

すでに同じ story がある場合は失敗します。

作り直しが必要なときだけ `--force-replace` を使ってください。

```

```bash

```

```
python -m tools.import_story stories/ankoku_gakuen --db db/pocketrole.db --
force-replace
```
```

``--force-replace`` は破壊的です。
進行中 story では通常使いません。

このとき ``web/assets/story\_maps/{story\_id}/`` の scaffold も作られます。
自動生成されるのは主に次です。

```
- `place_manifest.json`
- `images/`
- `bgm/`
- `bgm_manifest.json`
```

ただし、Place 背景画像、``map.json``、実際の MP3 は手動で配置します。

7-3. smoke 実行で軽く確認する

```
```bash
python -m tools.smoke_engine_start --story ankoku_gakuen --db db/pocketrole.db
--turns 1
```
```

このコマンドは ``/tmp`` の作業 DB で実行します。
本番 DB や public viewer の確認にはなりません。

8. 進行中 story を更新する手順

進行中 story の places や characters、schedule などを変えたいときは ``update\_story`` を使います。

```
```bash
python -m tools.update_story stories/ankoku_gakuen --db db/pocketrole.db
```
```

``update\_story`` の安全ルール

進行済み story では次の項目は guarded field です。

```
- `season_start`
- `turn_minutes`
```

これらを変えたい場合は、通常更新ではなく再構築が必要です。

9. engine を起動する

起動前に、必要なら ``config/llm\_runtime.local.yaml`` を目的の profile に合わせてください。
たとえば OpenAI で回したい日は ``active\_profile: "openai\_nano"``、ローカル ollama に戻したい日は ``active\_profile: "ollama\_local"`` のように切り替えます。

基本の起動:

```
```bash
python -m engine.main --stories ankoku_gakuen --db db/pocketrole.db --config
config.yaml
```
```

複数 story を同時に動かすこともできます。

```
```bash
python -m engine.main --stories ankoku_gakuen ankoku_gakuen_mystery --db
db/pocketrole.db --config config.yaml
```
```

ログレベルを変えたいとき:

```
```bash
python -m engine.main --stories ankoku_gakuen --log-level DEBUG
```
```

10. admin の local viewer で会話を見る

local viewer は、公開用 `viewer.php` ではなく local DB を読む admin 配下の閲覧画面です。初回成功確認では、ログや SQL だけでなく、まずこの画面で `ankoku\_gakuen` の会話と表情画像を見るのが分かりやすいです。

admin service を起動します。

```
```bash
POCKETROLE_ADMIN_USER="local_admin" \
POCKETROLE_ADMIN_PASSWORD="change-me-before-sharing" \
POCKETROLE_ADMIN_COOKIE_SECURE=0 \
python -m admin.main
```
```

ブラウザで開きます。

```
```text
http://127.0.0.1:8787/admin
```
```

上の例では、ログインユーザーは `local\_admin`、パスワードは `change-me-before-sharing` です。

admin service は `admin/admin` のままでは起動しません。

ログイン後、story 詳細から `viewer を開く` を選ぶか、次を直接開きます。

```
```text
http://127.0.0.1:8787/admin/viewer/ankoku_gakuen
```
```

local viewer では主に次を確認できます。

- runtime 状態
- 使用 provider / model
- latest turn
- キャラ名
- 表情画像
- 場所
- 発話本文

この画面は `db/pocketrole.db` を見えています。Web 投稿や公開 viewer の設定が未完了でも、local DB にログがあれば表示できます。

11. sample story から自作 story を作り始める

`ankoku\_gakuen` が動いたら、clone wizard で自分用 story を作り始められます。

```
```text
http://127.0.0.1:8787/admin/onboarding/ankoku_gakuen
```
```

この wizard は、sample story を複製して自分用 story を作り始める最初の橋です。
作成時に次の最小項目を差し替えます。

- 新しい `story\_id`
- タイトル
- 説明
- キャラ名
- キャラの短い説明
- goal / worry
- キャラごとの `neutral.png`

作成時には、内部で次の処理が実行されます。

1. `stories/<new\_story\_id>/` を作成
2. `world\_config.yaml` / `characters.yaml` の ID と基本項目を差し替え
3. character image assets と story map assets を複製
4. `validate\_story`
5. `import\_story`
6. `chapters.yaml` と `director.yaml` があれば追加 import
7. story metadata を生成

重複する `story\_id` は拒否されます。

通常 UI から `import\_story --force-replace` は使いません。

clone wizard で作った後は、admin の story 詳細から次の編集画面へ進めます。

- キャラ基本情報、追加、削除、ID rename、10 表情画像
- 場所基本情報、追加、削除、ID rename、隣接関係
- story 基本設定、実行設定、LLM runtime profile
- イベント/異変、condition builder
- director persona
- chapter / beat / beat event

細かい一括編集や高度な運用では、従来どおり YAML を編集し、`validate\_story` と `update\_story`、必要な追加 import を使ってください。

12. ログを見る

JSON ログは通常 `logs/engine.log` に出ます。

```
```bash
tail -f logs/engine.log | python -m json.tool
```
```

注目するとよい項目：

- `story\_id`
- `char\_id`

```
- `turn`  
- `llm_provider`  
- `llm_model`  
- `llm_latency_ms`  
- `llm_completion_status`
```

13. 長時間運転を監視する

実 DB で story quality を見るときは、engine の外から monitor を回すのが安全です。

```
```bash  
python -m tools.monitor_engine_runtime --story ankoku_gakuen --db
db/pocketrole.db --engine-pid <PID>
```
```

この monitor は次のような指標を Markdown に追記します。

```
- `reply_ratio`  
- `all_chars_spoke_ratio`  
- `open_hooks`  
- `active_tensions`  
- `growth_commits_recent`  
- `scene_arcs_recent`
```

warning も同時に見ます。

```
- `turn_stalled`  
- `reply_ratio_low`  
- `all_chars_spoke_high`  
- `tension_without_intervention`  
- `quality_retry_high`
```

story 性や出力品質の改善ポイントをまとめて見返したいときは、
`docs/specs/story-quality-playbook.md` を併用してください。

14. 公開 viewer に反映する

14-1. story ごとの Web 投稿設定を入れる

投稿先は `config/web\_post\_targets.local.yaml` に story ごとに設定します。

```
```yaml  
ankoku_gakuen:
 enabled: true
 receiver_url: "https://example.com/chururun/receiver.php"
 auth_token: "your-secret-token"
```
```

engine 起動中は `WebPoster` が `chat\_logs` の未送信分を `receiver.php` へ送ります。

14-2. viewer 側の確認方法

公開確認では、次を分けて見るのが安全です。

1. viewer ページ
2. `status.php`
3. `api.php?action=latest`

例:

```
```text
https://example.com/chururun/viewer.php?story_id=ankoku_gakuen
https://example.com/chururun/status.php
https://example.com/chururun/api.php?
story_id=ankoku_gakuen&action=latest&limit=5
```
```

14-3. `map\_replay.php` と BGM

保存済みログをシネマティックに再生するページは `map\_replay.php` です。

```
```text
https://example.com/chururun/map_replay.php?story_id=ankoku_gakuen
```
```

Place 背景画像は通常、次へ置きます。

- `web/assets/story\_maps/{story\_id}/images/{place\_id}\_320.png`

BGM は次の構成です。

- 既定 BGM: `web/assets/audio/pocketrole\_bgm.mp3`
- story 専用 BGM: `web/assets/story\_maps/{story\_id}/bgm/`
- story ごとの設定: `web/assets/story\_maps/{story\_id}/bgm\_manifest.json`

`bgm\_manifest.json` の最小例:

```
```json
{
 "enabled": true,
 "story_default": "bgm/ankoku_gakuen_theme.mp3",
 "place_overrides": {},
 "mood_overrides": {}
},,
```
```

現在の優先順位は次です。

1. `bgm\_manifest.json` の `story\_default`
2. `assets/audio/pocketrole\_bgm.mp3`
3. どちらも無ければ無音

BGM は `play` の最初のクリックで開始し、loop 再生されます。

`place\_overrides` と `mood\_overrides` は将来拡張用で、現行 `map\_replay` では `story\_default` だけを使います。

14-4. viewer 確認時の注意

- local DB にログがあっても viewer に出ないことがあります
- `smoke\_engine\_start` は viewer 確認には使えません
- `status.php` の `log\_count` は `.dat` の行数であり、viewer 表示件数と完全一致しないことがあります
- `latest` は公開 API ですが、`history` と `characters` は認証が必要です
- `map\_replay` の JS や BGM/画像を更新した直後は、ブラウザキャッシュで古い挙動が残ることがあります
- 反映直後に見え方や音が変わらない場合は、hard reload / private window / query 付き URL を先に試してください

15. ログと小説を書き出す

ログを書き出す

Markdown:

```
```bash
python -m tools.export_log --story ankoku_gakuen --format markdown --output
exports/ankoku_gakuen_log.md
```
```

テキスト:

```
```bash
python -m tools.export_log --story ankoku_gakuen --format text --output
exports/ankoku_gakuen_log.txt
```
```

小説を書き出す

`novel\_generator` と scene-aware prose は実装済みですが、実際に出力が増えるかは scene close や runtime 設定に依存します。
データがたまっていれば次のコマンドで書き出せます。

```
```bash
python -m tools.export_novel --story ankoku_gakuen --format markdown --output
exports/ankoku_gakuen_novel.md
```
```

review 下書きを作る

長時間運転の定量レビューは次で sidecar Markdown にできます。

```
```bash
python -m tools.generate_story_review_draft --story ankoku_gakuen --db
db/pocketrole.db --monitor-dir ../docs/monitoring
```
```

canonical review 台帳は自動では更新されません。

生成された draft を確認して、必要なら `docs/system\_review/` 側へ人手で取り込みます。

定量レビューをどう改善アクションへつなぐか迷ったときは、

`docs/specs/story-quality-playbook.md` の症状別診断表を参照してください。

Manual PDF を再生成する

`USER\_MANUAL.md` を更新したら、配布物の PDF も同じタイミングで再生成してください。

この環境では `pandoc/xelatex` 固定ではなく、使えるローカル HTML/PDF ツールチェーンで同期させるのが current 運用です。

この環境での再生成例:

```
```bash
env HOME=/tmp XDG_CACHE_HOME=/tmp XDG_CONFIG_HOME=/tmp XDG_RUNTIME_DIR=/tmp \
 libreoffice --headless --convert-to pdf --outdir /tmp
docs/users_manual/USER_MANUAL.md
cp /tmp/USER_MANUAL.pdf docs/users_manual/USER_MANUAL.pdf
```
```

最低限の確認:

- `docs/users\_manual/USER\_MANUAL.md` に新しい記述が入っている
- 再生成後の `docs/users\_manual/USER\_MANUAL.pdf` にも同じ語句が入っている
- 例: `local viewer`, `clone wizard`, `monitor\_engine\_runtime`, `generate\_story\_review\_draft`

キャラ成長 overlay を YAML に戻す

runtime で育った overlay は次で確認できます。

```
```bash
python -m tools.export_character_overlays --story ankoku_gakuen --db
db/pocketrole.db
```
```

必要なときだけ `characters.yaml` へ apply します。

```
```bash
python -m tools.apply_character_overlays --story-dir stories/ankoku_gakuen --
overlay stories/ankoku_gakuen/character_overlays.generated.yaml --output
/tmp/characters.merged.yaml
```
```

16. よくあるつまずき

`config.yaml` が見つからない

`config.yaml.example` をコピーして `config.yaml` を作ってください。

```
```bash
cp config.yaml.example config.yaml
```
```

`python engine/main.py` で import error になる

この project では直実行ではなく `python -m` 形式を使ってください。

```
```bash
python -m engine.main --stories ankoku_gakuen
```
```

`import\_story` と `update\_story` の使い分けが分からない

- 新規投入は `import\_story`
- 進行済み story の定義更新は `update\_story`
- 作り直しだけ `import\_story --force-replace`

smoke は通るのに public viewer が更新されない

正常です。

smoke は `/tmp` 作業 DB を使うため、public viewer の確認にはなりません。
実 DB を使った本起動と `WebPoster` の送信成功を確認してください。

admin の local viewer には出るのに public viewer に出ない

正常に起こります。

local viewer は `db/pocketrole.db` を直接読みますが、public viewer は `receiver.php` に送られた hosted 側ログを読みます。

公開したい場合は、`config/web\_post\_targets.local.yaml` と Web サーバー側 token を確認してください。

clone wizard で作った story をさらに細かく編集したい

clone wizard は最小導線です。

作成後の詳細編集は `stories/<story\_id>/world\_config.yaml` と `characters.yaml` を編集し、次を実行してください。

```
``bash
python -m tools.validate_story stories/<story_id>
python -m tools.update_story stories/<story_id> --db db/pocketrole.db
``
```

chapter / director を変更した場合は、追加 import も必要です。

新しい story-emergence 改修を入れたのに挙動が変わらない

まず `config.yaml` に feature flag セクションを足したか確認してください。

`config.yaml.example` をコピーしただけでは `scene\_management` や `growth\_engine` などが有効になっていないことがあります。

local DB にはあるのに viewer に出ない

次を順に確認してください。

1. `config/web\_post\_targets.local.yaml` の `receiver\_url` が正しいか
2. `config/web\_post\_targets.local.yaml` の `auth\_token` が正しいか
3. Web サーバー側 `web/config/auth\_token.txt` が同じ token になっているか
4. `logs/engine.log` に WebPoster 失敗が出ていないか
5. `status.php` で対象 story が見えているか
6. `api.php?action=latest` で最新ログが返るか

`DatabaseConnectionTimeoutError` が出る

DB 接続が時間内に完了していません。

次を確認してください。

1. 仮想環境が正しいか
2. `pip install -r requirements.txt` が完了しているか
3. `db/pocketrole.db` へのアクセス権があるか
4. 依存の `aiosqlite` が想定外の版になっていないか

17. 公開 viewer 利用者向けの短い説明

viewer 利用者が知っておけば十分なのは次の点です。

- story ごとに URL が分かります
- 画面はタイムライン中心です
- `最新へ` ボタンで下端へ移動できます

- 表示は local DB ではなく hosted 側の ``.dat`` を見えています

18. コマンド早見表

```
```bash
依存インストール
pip install -r requirements.txt

story 検証
python -m tools.validate_story stories/ankoku_gakuen

新規投入
python -m tools.import_story stories/ankoku_gakuen --db db/pocketrole.db

進行中 story 更新
python -m tools.update_story stories/ankoku_gakuen --db db/pocketrole.db

chapter / director 定義投入
python -m tools.import_chapters --story ankoku_gakuen --db db/pocketrole.db
python -m tools.import_director --story ankoku_gakuen --db db/pocketrole.db

Web 投稿先設定ファイルを初期化
python -m tools.init_web_post_targets --stories ankoku_gakuen

smoke 実行
python -m tools.smoke_engine_start --story ankoku_gakuen --db db/pocketrole.db
--turns 1

OpenAI model smoke 比較
python -m tools.smoke_openai_models --story ankoku_gakuen --db db/pocketrole.db
--models gpt-5-mini gpt-5.4-nano

長時間運転 monitor
python -m tools.monitor_engine_runtime --story ankoku_gakuen --db
db/pocketrole.db --engine-pid <PID>

review draft 生成
python -m tools.generate_story_review_draft --story ankoku_gakuen --db
db/pocketrole.db --monitor-dir ../docs/monitoring

overlay export
python -m tools.export_character_overlays --story ankoku_gakuen --db
db/pocketrole.db

overlay apply
python -m tools.apply_character_overlays --story-dir stories/ankoku_gakuen --
overlay stories/ankoku_gakuen/character_overlays.generated.yaml --output
/tmp/characters.merged.yaml

engine 起動
python -m engine.main --stories ankoku_gakuen --db db/pocketrole.db --config
config.yaml

admin 起動
POCKETROLE_ADMIN_USER="local_admin" \
POCKETROLE_ADMIN_PASSWORD="change-me-before-sharing" \
POCKETROLE_ADMIN_COOKIE_SECURE=0 \
python -m admin.main
```

```
ログ確認
tail -f logs/engine.log | python -m json.tool

ログ書き出し
python -m tools.export_log --story ankoku_gakuen --format markdown --output
exports/ankoku_gakuen_log.md

小説書き出し
python -m tools.export_novel --story ankoku_gakuen --format markdown --output
exports/ankoku_gakuen_novel.md
``,``
```

## ## 19. 最後に

運用の基本は次の順番です。

1. `validate\_story`
2. 新規なら `import\_story`
3. 進行済みなら `update\_story`
4. 必要なら `smoke\_engine\_start`
5. 本番は `engine.main`
6. 初回確認は `/admin/viewer/<story\_id>`
7. 公開確認は `status.php` と `latest` API も併用

加えて、story-emergence 機能を使う場合は feature flag を明示有効化し、  
`monitor\_engine\_runtime` と `generate\_story\_review\_draft` で実運転レビューを取ってください。

この流れを守ると、破壊的更新や「local では動いたのに viewer は出ない」「改修コードは入ったのに旧挙動のまま」といった混乱を減らせます。

## ## 20. hosted 管理画面を配備する

hosted 運用では、公開 PHP 面とは別に `aiohttp` の管理サービスを立てます。  
正本の構成は次です。

- 公開面
  - `viewer.php`
  - `api.php`
  - `archive.php`
  - `archive\_index.php`
  - `/published/\*`
- 管理面
  - `/admin/\*`
  - `127.0.0.1:8787` の admin service を reverse proxy する

### ### 20-1. env file を用意する

テンプレートは `deploy/examples/pocketrole-admin.env.example` です。  
少なくとも次を本番値に変えてください。

- `POCKETROLE\_ADMIN\_USER`
- `POCKETROLE\_ADMIN\_PASSWORD`
- `WEB\_AUTH\_TOKEN`

ここでいう `WEB\_AUTH\_TOKEN` は、hosted admin / deploy 用の env に置く値です。



ローカル engine の story ごとの投稿設定は `config/web\_post\_targets.local.yaml` を使います。

`admin/admin` のままでは管理サービスは起動しません。

### ### 20-2. systemd で起動する

参考 unit は `deploy/systemd/pocketrole-admin.service` です。

配置後の典型例:

```
```bash
sudo install -m 644 deploy/systemd/pocketrole-admin.service /etc/systemd/system/
sudo install -m 600 deploy/examples/pocketrole-admin.env.example
/etc/pocketrole/pocketrole-admin.env
sudo systemctl daemon-reload
sudo systemctl enable --now pocketrole-admin
```
```

### ### 20-3. reverse proxy を置く

参考設定は `deploy/nginx/pocketrole.conf.example` です。

- `/admin/` は `http://127.0.0.1:8787/admin/` へ proxy
- `/published/` は static 配信
- PHP 面は既存 `viewer.php` などをそのまま処理

### ### 20-4. health check

管理サービスの最低限の生存確認:

```
```bash
curl -sS http://127.0.0.1:8787/admin/healthz
```
```

### ### 20-5. AI エージェント用 token を発行する

```
```bash
python -m tools.admin_token create --db db/pocketrole.db --label hosted-agent
```
```

一覧:

```
```bash
python -m tools.admin_token list --db db/pocketrole.db
```
```

失効:

```
```bash
python -m tools.admin_token revoke --db db/pocketrole.db --token-id 1
```
```